

SIG-LOG

Sistema Integral de Gestión Logística

MANUAL TÉCNICO

Instalación reproducible, arquitectura, datos, ETL, modelos, análisis, dashboard, pruebas y mantenimiento

Proyecto

Programa educativo

Asignatura

Versión

SIG-LOG: Sistema Integral de Gestión Logística

Ingeniería en Desarrollo y Gestión de Software

Extracción del conocimiento en bases de datos

2.0 • Agosto 2026

Objetivo del documento

Permitir que una persona que no conoce SIG-LOG pueda comprender su propósito, instalarlo, ejecutarlo, consultar sus resultados y tomar decisiones con base en la evidencia mostrada.

1. INTRODUCCIÓN

SIG-LOG es un sistema integral de gestión y análisis logístico desarrollado para transformar información operativa en conocimiento útil para la toma de decisiones.

El sistema concentra información relacionada con:

- Clientes.
- Vehículos.
- Operadores.
- Rutas.
- Entregas.
- Combustible.
- Mantenimiento.
- Fallas.
- Riesgos.
- Patrones de operación.
- Modelos predictivos.

La finalidad del sistema no es únicamente mostrar información mediante gráficas, sino establecer una cadena completa de procesamiento:

Datos → preparación → almacenamiento → análisis → resultados → visualización → interpretación → decisión.

El proyecto utiliza una arquitectura organizada por capas, donde los datos originales pasan por procesos de preparación y posteriormente son utilizados por el Data Warehouse, los modelos analíticos y el dashboard.

La documentación existente del proyecto establece explícitamente esta separación entre fuentes, preparación, almacenamiento analítico, analítica y presentación.

2. PROPÓSITO DEL MANUAL

El presente documento tiene como finalidad explicar técnicamente cómo funciona SIG-LOG y cómo puede ser instalado, ejecutado, validado y mantenido.

Está dirigido principalmente a:

- Desarrolladores.
- Administradores del proyecto.
- Personal técnico.
- Evaluadores académicos.
- Personas que reciban el repositorio posteriormente.

- Personas responsables de actualizar datos o modelos.

A diferencia del Manual de Usuario, este documento se concentra en responder:

- ¿Cómo está construido SIG-LOG?
- ¿Qué componentes lo integran?
- ¿Dónde están los datos?
- ¿Cómo se transforman?
- ¿Dónde se almacenan?
- ¿Cómo se generan los resultados?
- ¿Dónde están los modelos?
- ¿Cómo se conecta el dashboard con el Data Warehouse?
- ¿Qué ocurre cuando faltan datos?
- ¿Cómo se diagnostican errores?
- ¿Cómo se actualiza el sistema?
- ¿Cómo puede reproducirse en otro equipo?

3. ALCANCE TÉCNICO

El manual cubre:

1. Arquitectura.
2. Estructura del repositorio.
3. Requisitos.
4. Instalación.
5. Configuración del entorno.
6. Ejecución.
7. Fuentes de datos.
8. ETL.
9. Calidad de datos.
10. Data Warehouse.
11. Consultas.
12. Modelos supervisados.
13. Clasificación.
14. Regresión.
15. K-Means.
16. PCA.
17. Mantenimiento.
18. Riesgos.
19. Dashboard.
20. Visualizaciones.
21. Exportación.
22. Manejo de errores.
23. Pruebas.

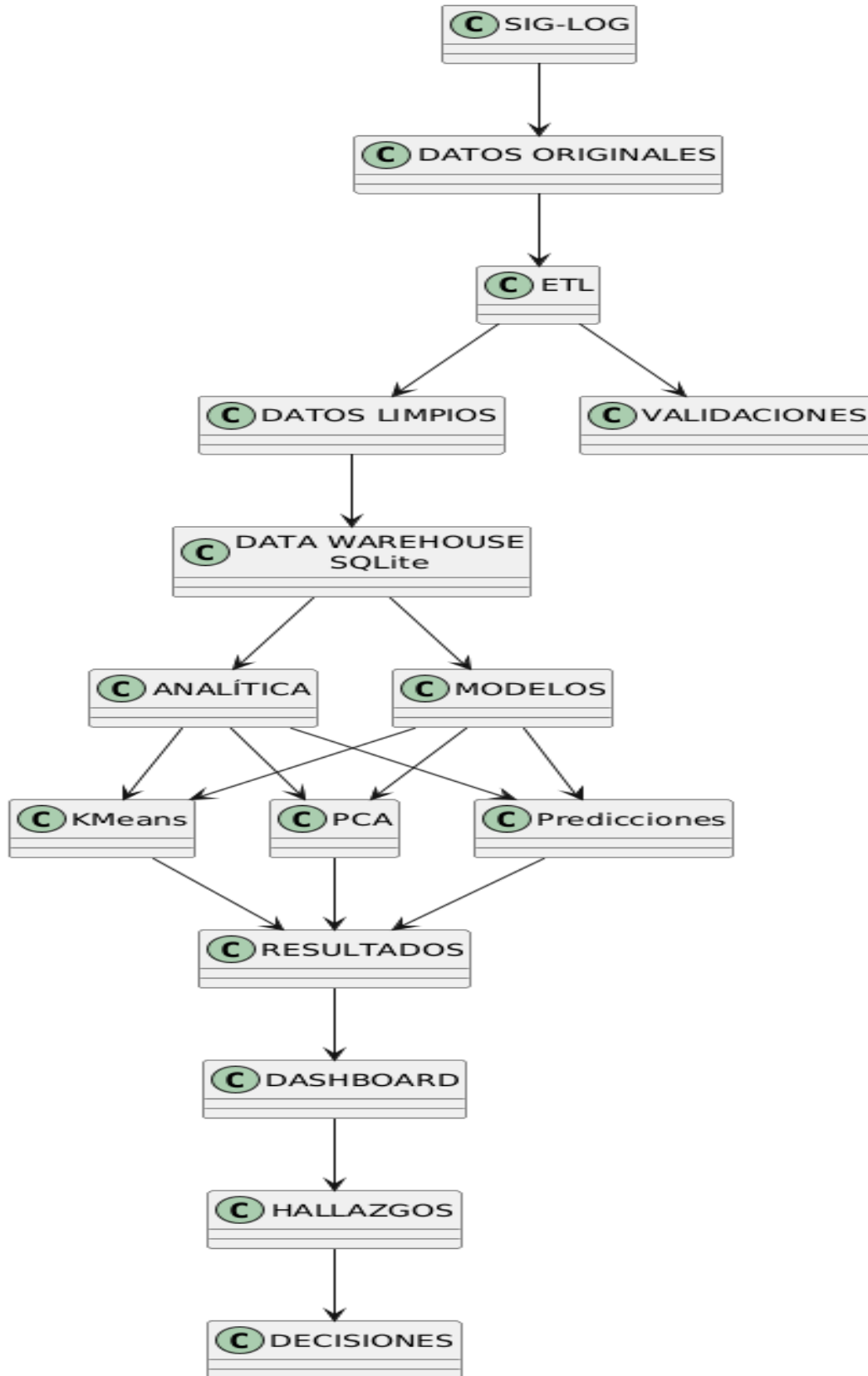
- 24. Respaldo.
- 25. Actualización.
- 26. Git.
- 27. Reproducibilidad.

4. DESCRIPCIÓN GENERAL DEL SISTEMA

SIG-LOG integra diferentes componentes en una sola solución.

La arquitectura funcional puede representarse de la siguiente manera:

Flujo de SIG-LOG



5. ARQUITECTURA POR CAPAS

SIG-LOG se organiza en cinco capas principales.

Capa	Componentes	Responsabilidad
1. Fuentes	CSV originales	Contener los registros operativos
2. Preparación	etl/, datos/limpios/	Limpiar, transformar y preparar
3. Almacenamiento	data_warehouse/	Centralizar información analítica
4. Analítica	modelos/, mantenimiento/, no_supervisado/, pca/	Generar modelos, patrones y resultados
5. Presentación	dashboard/, graficas/, reportes/	Presentar resultados y evidencias

Esta organización corresponde a la estructura documentada del proyecto.

6. PRINCIPIO DE DEPENDENCIA ENTRE CAPAS

Las capas presentan una dependencia lógica.

Por ejemplo:

CSV

↓

ETL

↓

Datos limpios

↓

Data Warehouse

↓

Consultas

↓

Modelos / análisis

↓

Resultados



Dashboard

Por lo tanto, una modificación en una capa inferior puede afectar a las capas posteriores.

Ejemplo:

Cambio de columna en entregas.csv



Cambio en ETL



Cambio en Data Warehouse



Cambio en consulta



Cambio en modelo



Cambio en dashboard

Por esta razón, los cambios estructurales deben realizarse de forma controlada.

7. ESTRUCTURA DEL PROYECTO

La estructura documentada de SIG-LOG contempla:

SIG-LOG/



|— clientes.csv

|— componentes.csv

|— entregas.csv

|— operadores.csv

|— rutas.csv

|— vehiculos.csv



|— dashboard/

| |— dashboard.py



```
|— datos/
|   |— limpios/
|
|— etl/
|   |— cargar_datos.py
|   |— limpiar_datos.py
|   |— transformar_datos.py
|
|— data_warehouse/
|   |— crear_warehouse.py
|   |— cargar_warehouse.py
|   |— consultas_warehouse.py
|   |— siglog_dw.db
|
|— modelos/
|   |— clasificacion.py
|   |— regresion.py
|   |— entrenamiento.py
|   |— optimizacion.py
|
|— modelos_entrenados/
|   |— DecisionTree.pkl
|   |— RandomForest.pkl
|   |— LogisticRegression.pkl
|   |— LinearRegression.pkl
|   |— DecisionTreeRegressor.pkl
|   |— RandomForestRegressor.pkl
|   |— metadata_entrenamiento.json
|   |— preprocesador_entregas.pkl
|
|— mantenimiento/
|   |— scripts
|   |— resultados/
```

```
|
|
|— no_supervisado/
|   |— kmeans.py
|   |— analizar_clusters.py
|   |— resultados/
|
|
|— pca/
|   |— procesos
|   |— resultados/
|
|
|— graficas/
|
|
|— reportes/
|
|
|— reportes_modelos/
```

La documentación del proyecto confirma esta organización y establece las responsabilidades de cada una de estas rutas.

8. INVENTARIO TÉCNICO

8.1 Dashboard

dashboard\dashboard.py

Es la aplicación principal de Streamlit.

Responsabilidades:

- Presentación.
- Navegación.
- Consulta de datos.
- Filtros.
- Indicadores.
- Gráficas.
- Predicciones.
- Análisis avanzado.
- Reportes.
- Descargas.

El código actual establece como raíz del proyecto la carpeta superior a **dashboard** y localiza directamente:

```
data_warehouse\siglog_dw.db  
modelos_entrenados\  
no_supervisado\resultados\  
pca\resultados\  
mantenimiento\resultados\  
reportes_modelos\  

```

Esto está implementado directamente en el dashboard.

9. MÓDULOS ACTUALES DEL DASHBOARD

El dashboard implementado contiene los siguientes módulos:

1. Resumen.
2. Entregas.
3. Clientes.
4. Vehículos.
5. Operadores.
6. Rutas.
7. Combustible.
8. Asignaciones.
9. Mantenimiento.
10. Riesgos y alertas.
11. Preguntas de negocio.
12. Patrones de operación.
13. Análisis avanzado.
14. Mapa operativo.
15. Reportes.
16. Datos.

La navegación está implementada directamente mediante el menú lateral del dashboard.

10. REQUISITOS TÉCNICOS

Hardware recomendado

- Procesador moderno.
- 8 GB de RAM o superior.
- Espacio suficiente para datos, modelos y resultados.
- Pantalla con resolución adecuada para el dashboard.

Software

- Windows 10/11 o plataforma compatible.
- Python.
- Conda/Anaconda/Miniconda, preferentemente.
- PowerShell o terminal equivalente.
- Navegador moderno.
- Git, recomendado para control de versiones.

Estos requisitos están contemplados en la documentación técnica existente.

11. DEPENDENCIAS

Las principales bibliotecas utilizadas son:

streamlit

pandas

numpy

plotly

scikit-learn

joblib

matplotlib

La instalación comprobada del proyecto contempla:

`pip install streamlit pandas numpy plotly scikit-learn joblib matplotlib`

12. INSTALACIÓN DESDE CERO

Paso 1. Abrir terminal

Abrir:

- PowerShell.
- Anaconda Prompt.
- Terminal compatible.

Paso 2. Entrar a la raíz

Se debe trabajar desde la carpeta raíz:

`SIG-LOG\`

No se recomienda ejecutar los comandos desde:

SIG-LOG\dashboard\

La documentación de instalación específica ejecutar los comandos desde la raíz del proyecto.

13. CREACIÓN DEL ENTORNO

Ejecutar:

```
conda create -n siglog python=3.13 -y
```

Activar:

```
conda activate siglog
```

Estos son los comandos establecidos para la instalación reproducible del proyecto.

14. INSTALACIÓN DE DEPENDENCIAS

Ejecutar:

```
pip install streamlit pandas numpy plotly scikit-learn joblib matplotlib
```

Si el repositorio contiene [requirements.txt](#), se recomienda utilizar:

```
pip install -r requirements.txt
```

La documentación del proyecto recomienda congelar las dependencias funcionales para permitir la reproducción en otro equipo.

15. VERIFICACIÓN DE PYTHON

Ejecutar:

```
python --version
```

Debe aparecer la versión instalada.

16. VERIFICACIÓN DE DEPENDENCIAS

Ejecutar:

```
pip show streamlit
```

```
pip show pandas
```

```
pip show numpy
```

```
pip show plotly
```

```
pip show scikit-learn
```

`pip show joblib`

`pip show matplotlib`

También puede utilizarse:

`pip list`

17. COMPROBACIÓN DE LA ESTRUCTURA

Antes de iniciar el sistema se debe comprobar la existencia de:

`clientes.csv`

`componentes.csv`

`entregas.csv`

`operadores.csv`

`rutas.csv`

`vehiculos.csv`

Además de:

`datos\limpios\`

`data_warehouse\siglog_dw.db`

`modelos_entrenados\`

`mantenimiento\resultados\`

`no_supervisado\resultados\`

`pca\resultados\`

`reportes_modelos\`

`dashboard\dashboard.py`

Esta estructura está documentada como la estructura esperada del proyecto.

18. EJECUCIÓN DEL SISTEMA

Desde la raíz del proyecto:

`streamlit run dashboard\dashboard.py`

La ejecución comprobada del proyecto utiliza exactamente este punto de entrada.

Cuando Streamlit inicia correctamente se utiliza:

`http://localhost:8501`

El Manual de Usuario del proyecto documenta esta dirección de ejecución local.

19. DETENER EL SISTEMA

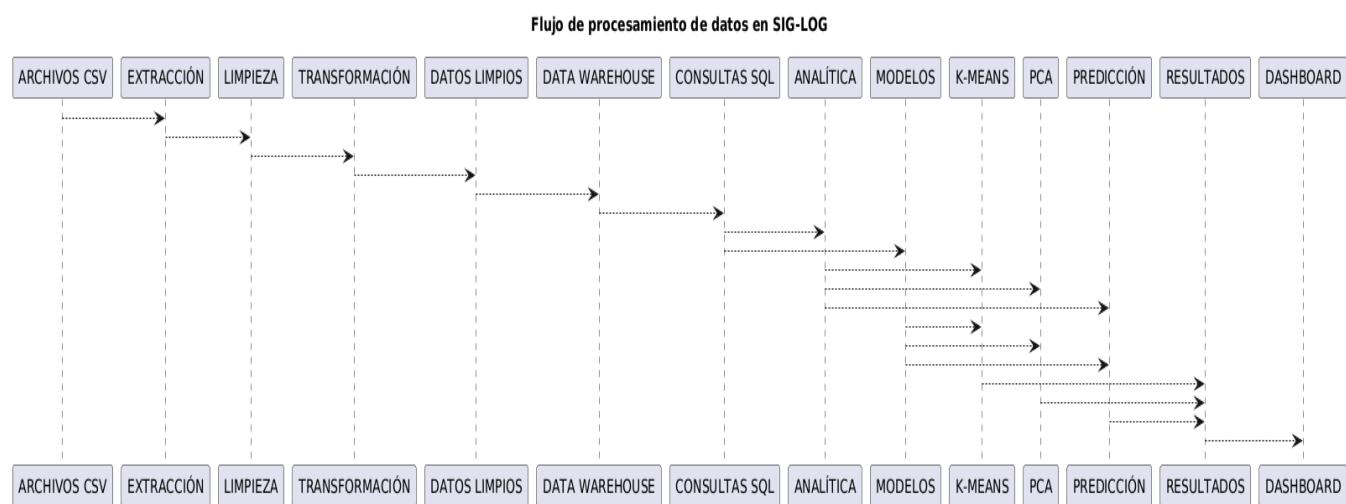
En la terminal donde está ejecutándose Streamlit:

Ctrl + C

Esto detiene el servidor local.

20. FLUJO DE DATOS

El flujo técnico de SIG-LOG es:



21. FUENTES DE INFORMACIÓN

Las fuentes principales son:

Archivo	Entidad	Uso
clientes.csv	Clientes	Demanda y servicios
vehiculos.csv	Vehículos	Operación, costos y mantenimiento
operadores.csv	Operadores	Asignación de entregas
rutas.csv	Rutas	Distancia, frecuencia y retrasos
entregas.csv	Entregas	Cumplimiento del servicio

componentes.csv	Componentes	Fallas y mantenimiento
-----------------	-------------	------------------------

Además, la estructura actual contempla **mantenimiento.csv** dentro de los datos preparados/documentados.

22. ETL

ETL significa:

Extract

Transform

Load

En español:

Extraer

Transformar

Cargar

En SIG-LOG el ETL prepara los datos antes de utilizarlos en el análisis.

23. EXTRACCIÓN

La extracción consiste en obtener los registros desde las fuentes originales.

Durante esta etapa se debe comprobar:

- Existencia del archivo.
- Nombre.
- Ruta.
- Número de registros.
- Número de columnas.
- Nombres de las variables.

La documentación técnica establece explícitamente estos controles.

24. LIMPIEZA

El proceso de limpieza contempla, según las reglas documentadas:

- Normalización de columnas.
- Conversión de tipos.
- Fechas.
- Valores nulos.

- Duplicados.
- Variables numéricas.
- Variables categóricas.
- Homologación de nombres.
- Variables derivadas.

25. DATOS LIMPIOS

Los datos procesados se almacenan en:

datos\limpios\

Esta carpeta representa la salida de la etapa de preparación.

No debe confundirse con los CSV originales.

26. REGLA PARA LOS DATOS ORIGINALES

Durante una demostración o validación:

No modificar directamente los CSV originales.

Si se requiere experimentar:

1. Crear una copia.
2. Realizar cambios.
3. Ejecutar nuevamente el proceso.
4. Validar resultados.

Esta recomendación también forma parte de las buenas prácticas documentadas del proyecto.

27. CALIDAD DE DATOS

Los controles técnicos principales son:

Existencia

¿Existe el archivo?

Estructura

¿Existen las columnas requeridas?

Tipos

¿Las columnas contienen datos compatibles?

Nulos

¿Existen valores faltantes?

Duplicados

¿Existen registros repetidos?

Rango

¿Los valores son razonables?

Integridad

¿Las relaciones entre entidades son coherentes?

28. DATA WAREHOUSE

SIG-LOG utiliza:

SQLite

como almacenamiento analítico.

Archivo:

data_warehouse\siglog_dw.db

La base funciona como punto central para las consultas utilizadas por la aplicación.

29. COMPONENTES DEL DATA WAREHOUSE

crear_warehouse.py

Responsable de crear la estructura.

cargar_warehouse.py

Responsable de cargar los datos.

consultas_warehouse.py

Contiene consultas utilizadas para obtener información analítica.

siglog_dw.db

Es la base SQLite utilizada por el sistema.

La documentación técnica confirma estas responsabilidades.

30. CONEXIÓN DEL DASHBOARD CON EL WAREHOUSE

El dashboard obtiene la ruta del Data Warehouse mediante:

`data_warehouse\siglog_dw.db`

El código actual establece esta ubicación de manera relativa a la raíz del proyecto.

Además, el dashboard comprueba que la base exista.

Si no existe:

No se encontró el Data Warehouse.

y la aplicación detiene la ejecución.

31. CONSULTAS ANALÍTICAS

Las consultas SQL permiten obtener información agregada y relacionada.

Por ejemplo, el dashboard consulta información de mantenimiento relacionando:

`dim_vehiculo`

con:

`fact_mantenimiento`

mediante:

`id_vehiculo`

El resultado incluye:

- Mantenimientos.
- Fallas.
- Emergencias.
- Fallas críticas.
- Costo de mantenimiento.
- Horas fuera de servicio.

Esto está implementado en el código actual del dashboard.

32. VALIDACIÓN DEL WAREHOUSE

Antes de ejecutar el dashboard debe comprobarse:

¿Existe `siglog_dw.db`?



Sí → continuar



No → corregir Warehouse

También deben comprobarse las tablas requeridas por las consultas.

33. MODELOS SUPERVISADOS

SIG-LOG contiene procesos de:

- Clasificación.
- Regresión.
- Entrenamiento.
- Optimización.

La carpeta:

modelos\

contiene los scripts correspondientes.

La carpeta:

modelos_entrenados\

contiene los artefactos serializados.

34. MODELOS ENTRENADOS

Los artefactos documentados incluyen:

DecisionTree.pkl

RandomForest.pkl

LogisticRegression.pkl

LinearRegression.pkl

DecisionTreeRegressor.pkl

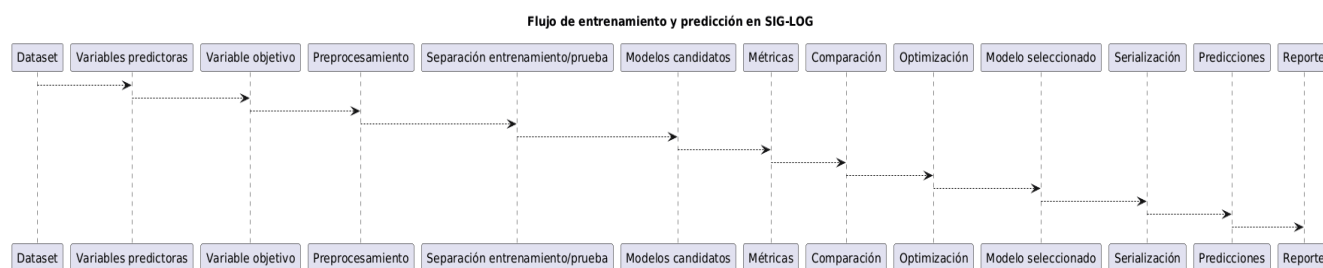
RandomForestRegressor.pkl

metadata_entrenamiento.json

preprocesador_entregas.pkl

35. FLUJO DE ENTRENAMIENTO

El flujo establecido es:



Este flujo está documentado en el manual técnico actual.

36. CLASIFICACIÓN

La clasificación busca asignar registros a categorías.

En SIG-LOG se utiliza para analizar escenarios como el riesgo asociado a entregas.

Conceptualmente:

Características de entrega



Modelo



Categoría / riesgo

37. REGRESIÓN

La regresión busca estimar valores numéricos.

Conceptualmente:

Variables de entrada



Modelo



Valor estimado

38. EVALUACIÓN DE MODELOS

Para clasificación pueden utilizarse:

- Exactitud.
- Precisión.
- Recall.

- Matriz de confusión.

Para regresión:

- MAE.
- MSE.
- RMSE.

Además, debe compararse el comportamiento entre entrenamiento y prueba para detectar sobreajuste.

39. PREPROCESADOR

SIG-LOG conserva:

preprocesador_entregas.pkl

Esto permite aplicar el mismo tratamiento a los datos utilizados durante la predicción.

La regla técnica es:

Datos

↓

Mismo preprocesamiento

↓

Modelo entrenado

↓

Predicción

40. SERIALIZACIÓN

Los modelos se almacenan como archivos:

.pkl

Esto permite cargar el modelo sin entrenarlo nuevamente.

Los archivos serializados no deben modificarse manualmente.

41. ANÁLISIS NO SUPERVISADO

SIG-LOG incorpora análisis mediante:

K-Means

La carpeta correspondiente es:

no_supervisado\

Los procesos principales documentados son:

kmeans.py

analizar_clusters.py

42. RESULTADOS DE K-MEANS

Los resultados contemplados son:

entregas_clusters.csv

evaluacion_kmeans.csv

perfil_clusters.csv

Cada archivo tiene una función:

Archivo	Función
entregas_clusters.csv	Asignación de registros a grupos
evaluacion_kmeans.csv	Evaluación de agrupación
perfil_clusters.csv	Características de cada grupo

43. INTERPRETACIÓN DE CLUSTERS

Un cluster no significa automáticamente:

Bueno

Malo

Representa registros con características similares de acuerdo con las variables utilizadas.

La interpretación debe realizarse después del perfilamiento.

44. PCA

SIG-LOG incorpora análisis de reducción de dimensionalidad mediante:

PCA – Principal Component Analysis

Los resultados contemplados incluyen:

cargas_componentes.csv

entregas_pca.csv

interpretacion_componentes.csv

varianza_pca.csv

Estos resultados permiten estudiar:

- Variabilidad conservada.
- Componentes principales.
- Contribución de variables.
- Representación transformada.

45. MANTENIMIENTO

El módulo de mantenimiento analiza información relacionada con:

- Tipos de mantenimiento.
- Fallas.
- Componentes.
- Causas.
- Vehículos.
- Costos.
- Severidad.
- Emergencias.
- Tiempo fuera de servicio.

Los resultados del módulo se almacenan en:

mantenimiento\resultados\

46. ANÁLISIS DE RIESGOS

El dashboard calcula información relacionada con:

- Mantenimientos.
- Fallas.
- Emergencias.
- Fallas críticas.
- Costos.
- Horas fuera de servicio.
- Nivel de riesgo.

La consulta actual realiza estas agregaciones directamente sobre el Data Warehouse.

47. NIVELES DE RIESGO

Los niveles utilizados son:

BAJO

MEDIO

ALTO

CRÍTICO

Estos niveles funcionan como señales de priorización.

Un nivel de riesgo no debe interpretarse como un diagnóstico mecánico definitivo.

El Manual de Usuario establece expresamente esta limitación.

48. DASHBOARD

El dashboard es la capa de presentación de SIG-LOG.

Su configuración actual incluye:

page_title = SIG-LOG | Dashboard Ejecutivo

layout = wide

sidebar = expanded

y utiliza Plotly para las visualizaciones.

49. TECNOLOGÍAS DEL DASHBOARD

El código actual utiliza:

```
import numpy as np
```

```
import pandas as pd
```

```
import plotly.express as px
```

```
import plotly.graph_objects as go
```

```
import streamlit as st
```

Además utiliza:

```
import sqlite3
```

```
import json
```

```
import re
```

```
from pathlib import Path
```

entre otras bibliotecas necesarias para reportes y utilidades.

50. FILTROS

El dashboard trabaja con filtros para establecer el alcance del análisis.

Los filtros documentados incluyen:

- Año.
- Mes.
- Estado.
- Vehículo.
- Riesgo.

Los filtros deben considerarse antes de interpretar cualquier indicador.

El Manual de Usuario establece como regla:

No interpretar una cifra sin revisar periodo, filtros y número de registros.

51. VISUALIZACIONES

Las gráficas utilizan Plotly.

El diseño actual contempla:

- Interacción.
- Zoom.
- Selección.
- Tooltip.
- Títulos.
- Subtítulos.
- Periodos visibles.

El código del dashboard implementa una función común para construir las tarjetas de gráficas y mantener una presentación homogénea.

52. HOVER Y TOOLTIP

Las gráficas permiten pasar el cursor sobre los elementos.

El detalle puede incluir:

- Entidad.
- Valor.
- Porcentaje.
- Entregas.
- Tardías.
- Costo.

- Riesgo.

Esta funcionalidad está documentada tanto en el Manual de Usuario como en la implementación del dashboard.

53. ESTRUCTURA DE UNA GRÁFICA

Cada visualización debe responder una pregunta de negocio.

La estructura es:

TÍTULO



SUBTÍTULO



PERIODO



GRÁFICA



DETALLE / HOVER



INTERPRETACIÓN



ACCIÓN POSIBLE

La implementación del dashboard incorpora título, pregunta, descripción, periodo, gráfica e interpretación/recomendación.

54. PREGUNTAS DE NEGOCIO

El dashboard contiene lógica para relacionar títulos de gráficas con preguntas de negocio.

Entre las preguntas contempladas se encuentran:

- ¿Qué rutas son más utilizadas?
- ¿Qué vehículos generan mayores costos?
- ¿Qué vehículos consumen más combustible?
- ¿Qué operadores realizan más entregas?
- ¿Qué rutas presentan mayores retrasos?
- ¿Qué vehículos requieren mantenimiento?
- ¿Podemos identificar grupos de rutas similares?

- ¿Qué factores explican el comportamiento?
- ¿Es posible predecir si una entrega llegará tarde?
- ¿Cuáles son las causas principales de retraso?

Esto se encuentra implementado directamente en [dashboard.py](#).

55. DATOS VACÍOS

El dashboard controla explícitamente la ausencia de información.

Por ejemplo, si no se cargan entregas:

No se pudieron cargar las entregas.

y la ejecución se detiene para evitar continuar con información incompleta.

56. DATA WAREHOUSE NO DISPONIBLE

Si no existe:

`data_warehouse\siglog_dw.db`

el dashboard muestra un error y detiene la ejecución.

Esto evita mostrar información aparentemente válida cuando la fuente analítica no está disponible.

57. REPORTE

SIG-LOG contempla generación de reportes.

El código actual utiliza componentes de ReportLab y puede generar archivos PDF con información representada y fecha de generación.

58. EXPORTACIÓN CSV

Las gráficas pueden generar información exportable a CSV.

La implementación contempla convertir los datos de la gráfica en:

CSV UTF-8

cuando existe un DataFrame asociado.

59. TRAZABILIDAD

Cada resultado importante debe poder seguirse hacia su origen.

Resultado



Gráfica



DataFrame



Consulta



Warehouse



Datos preparados



Fuente

Esto permite responder técnicamente:

¿De dónde salió este número?

60. REPRODUCIBILIDAD

La reproducibilidad significa que otra persona puede:

1. Obtener el proyecto.
2. Crear el entorno.
3. Instalar dependencias.
4. Disponer de los datos.
5. Ejecutar los procesos.
6. Iniciar el dashboard.
7. Obtener resultados equivalentes.

Para facilitar esto se recomienda conservar:

Código

+

Datos

+

Dependencias

+

Modelos

+

Resultados

+

Documentación

61. PRUEBAS TÉCNICAS

La validación del sistema debe considerar:

Prueba de instalación

Entorno → Dependencias → Ejecución

Prueba de datos

Archivos → Columnas → Tipos → Valores

Prueba del Warehouse

Base → Tablas → Consultas

Prueba de modelos

Modelo → Carga → Predicción

Prueba del dashboard

Carga → Filtros → Gráficas → Descargas

63. ERRORES FRECUENTES

Error	Causa probable	Solución
ModuleNotFoundError	Falta biblioteca	Instalar dependencia
FileNotFoundError	Ruta incorrecta	Revisar estructura
KeyError	Falta columna	Revisar dataset
Gráfica vacía	No existen registros	Revisar filtros
No aparecen modelos	Faltan .pkl	Revisar modelos_entrenados\

No aparece información	Problema de ruta/datos	Revisar estructura
Puerto ocupado	Otra instancia	Cerrar proceso o cambiar puerto
<code>NameError: name 're' is not defined</code>	Falta importación	Agregar <code>import re</code>

Estos errores están registrados en la documentación del proyecto.

64. ERROR `NameError: name 're' is not defined`

Si aparece:

`NameError: name 're' is not defined`

se debe comprobar que `dashboard.py` tenga:

```
import re
```

La versión actual del archivo ya contiene esta importación.

65. PUERTO 8501 OCUPADO

Si el puerto está ocupado puede ejecutarse:

```
streamlit run dashboard\dashboard.py --server.port 8502
```

Entonces se accederá mediante:

`http://localhost:8502`

66. LECTURA DE UN TRACEBACK

Cuando aparezca un error:

1. Leer la última excepción.
2. Identificar el archivo.
3. Identificar la línea.
4. Leer el mensaje final.
5. Determinar la causa.
6. Corregir.
7. Guardar.
8. Recargar.

9. Probar nuevamente.

La documentación del proyecto recomienda concentrarse en la última excepción del traceback.

67. ACTUALIZACIÓN DE DATOS

Cuando se incorporen datos nuevos:

1. Respalidar datos actuales



2. Incorporar nuevos datos



3. Validar estructura



4. Ejecutar ETL



5. Revisar datos limpios



6. Actualizar Warehouse



7. Ejecutar análisis



8. Validar resultados



9. Probar dashboard

68. ACTUALIZACIÓN DE MODELOS

No se debe reemplazar un modelo únicamente porque existan nuevos datos.

Primero se debe:

1. Analizar el nuevo dataset.
2. Revisar variables.
3. Revisar variable objetivo.
4. Comparar métricas.
5. Validar entrenamiento/prueba.
6. Revisar comportamiento.

7. Guardar nueva versión.
8. Validar dashboard.

69. VERSIONADO DE MODELOS

Se recomienda conservar:

Modelo

+

Dataset utilizado

+

Fecha

+

Preprocesador

+

Métricas

+

Metadatos

Esto facilita conocer qué versión produjo una predicción determinada.

70. RESPALDOS

Antes de modificar elementos críticos:

RESPALDAR

☐ CSV

☐ datos\limpios\

☐ siglog_dw.db

☐ modelos_entrenados\

☐ resultados

☐ reportes

Nunca debe realizarse una modificación estructural importante sin respaldo.

71. GIT

El proyecto debe controlarse mediante Git.

Comprobar estado:

git status

Ver cambios:

git diff

Agregar:

git add .

Registrar:

git commit -m "Actualización SIG-LOG"

Enviar:

git push

72. ARCHIVOS SENSIBLES

No deben publicarse accidentalmente:

- Contraseñas.
- Tokens.
- Claves privadas.
- Credenciales.
- Archivos `.env` con información sensible.

Debe utilizarse:

`.gitignore`

cuando corresponda.

73. MANTENIMIENTO PREVENTIVO

Se recomienda revisar periódicamente:

- Dependencias.
- Python.
- Estructura de archivos.
- Data Warehouse.
- Modelos.
- Resultados.
- Dashboard.
- Consultas.
- Gráficas.

No se recomienda actualizar todas las bibliotecas sin realizar pruebas posteriores.

74. SEPARACIÓN DE RESPONSABILIDADES

La arquitectura debe conservar:

ETL



Datos preparados



Warehouse



Analítica



Resultados



Dashboard

No debe trasladarse toda la lógica al dashboard.

El dashboard debe principalmente:

- Consultar.
- Filtrar.
- Visualizar.
- Interpretar.
- Exportar.

75. PRINCIPIO DE NO INVENCION

Los resultados deben distinguir:

Dato

Información calculada directamente.

Interpretación

Explicación del resultado.

Recomendación

Acción que puede evaluarse.

Ejemplo:

Dato:

29 % de entregas tardías.

Interpretación:

Aproximadamente 3 de cada 10 entregas presentan retraso.

Recomendación:

Revisar las rutas y horarios con mayor concentración.

Una recomendación no debe presentarse como si fuera un dato.

76. PREDICCIONES

Las predicciones deben interpretarse como señales.

No debe decirse:

"La entrega definitivamente se retrasará."

Debe decirse:

"El modelo identifica características asociadas con un mayor riesgo de retraso."

El Manual de Usuario establece expresamente que una predicción no implica que el evento ocurrirá con certeza.

77. MANTENIMIENTO Y DECISIONES

Los resultados de mantenimiento pueden utilizarse para priorizar investigaciones.

Ejemplo:

Falla recurrente



Componente



Causa registrada



Vehículo



Costo



Prioridad

Sin embargo, la confirmación física de una falla corresponde al área técnica.

78. K-MEANS Y DECISIONES

K-Means puede identificar grupos similares.

Ejemplo:

Grupo 1

Alta frecuencia

Grupo 2

Mayor retraso

Grupo 3

Comportamiento intermedio

La interpretación debe basarse en el perfil real de cada grupo.

79. PCA Y DECISIONES

PCA permite estudiar dimensiones principales del comportamiento.

No debe presentarse al usuario final únicamente como una explicación matemática.

Debe traducirse a conceptos de negocio:

Variables



Componentes



Patrones principales



Interpretación logística

80. CRITERIO DE CALIDAD DE UNA GRÁFICA

Una gráfica correcta debe permitir responder:

1. ¿Qué estoy viendo?
2. ¿Qué periodo representa?
3. ¿Qué unidad utiliza?
4. ¿Cuánto representa?
5. ¿Qué elemento destaca?
6. ¿Qué significa?
7. ¿Qué debería investigarse?

El Manual de Usuario establece que una visualización debe poder responder estas preguntas rápidamente.

81. CRITERIO DE CALIDAD DE LOS KPIs

Los indicadores deben evitar confundir:

Volumen

con:

Desempeño

Por ejemplo, una unidad puede presentar más entregas tardías absolutas simplemente porque realizó muchas más entregas.

Por eso deben utilizarse:

- Cantidades.
- Porcentajes.
- Contexto.
- Periodo.
- Número de registros.

82. EVIDENCIAS TÉCNICAS

Para la entrega académica se recomienda conservar:

Capturas del dashboard

+

Resultados de modelos

+

Resultados de mantenimiento

+

Resultados de K-Means

+

Resultados PCA

+

Datos limpios

+

Data Warehouse

Las rutas documentadas incluyen:

reportes_modelos\

mantenimiento\resultados\

no_supervisado\resultados\

pca\resultados\

datos\limpios\

data_warehouse\siglog_dw.db

83. CHECKLIST DE INSTALACIÓN COMPROBADA

- ☐ Proyecto SIG-LOG recibido
- ☐ Carpeta raíz identificada
- ☐ dashboard\dashboard.py presente
- ☐ Python instalado
- ☐ Entorno siglog creado
- ☐ Entorno activado
- ☐ Dependencias instaladas
- ☐ Archivos CSV presentes
- ☐ Datos limpios presentes
- ☐ Data Warehouse presente
- ☐ Modelos presentes
- ☐ Resultados presentes
- ☐ Streamlit ejecutado
- ☐ Dashboard disponible

84. CHECKLIST DE FUNCIONAMIENTO

- ☐ Dashboard inicia

- ☐ Resumen funciona
- ☐ Entregas funciona
- ☐ Clientes funciona
- ☐ Vehículos funciona
- ☐ Operadores funciona
- ☐ Rutas funciona
- ☐ Combustible funciona
- ☐ Asignaciones funciona
- ☐ Mantenimiento funciona
- ☐ Riesgos funciona
- ☐ Preguntas de negocio funcionan
- ☐ Patrones funcionan
- ☐ Análisis avanzado funciona
- ☐ Mapa funciona
- ☐ Reportes funcionan
- ☐ Datos funcionan

Los módulos enumerados corresponden a la navegación actualmente implementada en el dashboard.

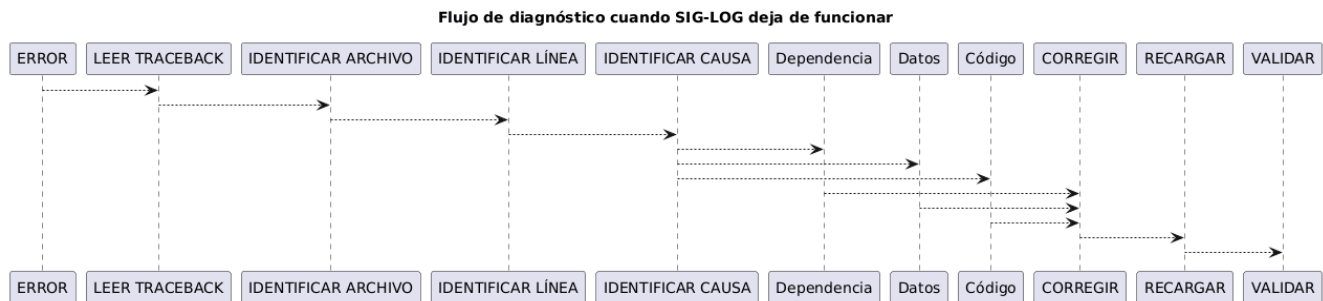
85. CHECKLIST DE ANÁLISIS AVANZADO

- ☐ Modelos entrenados disponibles
- ☐ Preprocesador disponible
- ☐ Clasificación disponible
- ☐ Regresión disponible
- ☐ Predicciones disponibles
- ☐ K-Means disponible
- ☐ Perfil de clusters disponible
- ☐ Evaluación K-Means disponible
- ☐ PCA disponible
- ☐ Varianza PCA disponible
- ☐ Cargas PCA disponibles

86. CHECKLIST DE MANTENIMIENTO

- ☐ Tipos de mantenimiento
- ☐ Fallas
- ☐ Componentes
- ☐ Causas
- ☐ Severidad
- ☐ Costos
- ☐ Emergencias
- ☐ Horas fuera de servicio
- ☐ Riesgo
- ☐ Resultados exportables

87. PROCEDIMIENTO DE RECUPERACIÓN



88. INSTALACIÓN EN UN EQUIPO NUEVO

Procedimiento completo:

```

git clone URL_DEL_REPOSITORIO
cd SIG-LOG
conda create -n siglog python=3.13 -y
conda activate siglog
pip install -r requirements.txt
streamlit run dashboard\dashboard.py
  
```

Si no existe **requirements.txt**:

```

pip install streamlit pandas numpy plotly scikit-learn joblib matplotlib
  
```

89. CRITERIO DE REPRODUCIBILIDAD

El proyecto se considera correctamente reproducible cuando una persona diferente al desarrollador puede:

1. Recibir el proyecto.
2. Crear el entorno.
3. Instalar las dependencias.
4. Encontrar los datos.
5. Encontrar el Warehouse.
6. Encontrar los modelos.
7. Ejecutar el dashboard.
8. Consultar la información.
9. Obtener resultados.
10. Diagnosticar problemas básicos.

90. PREGUNTAS DE DEFENSA TÉCNICA

¿Qué problema resuelve SIG-LOG?

Integra información logística para analizar entregas, rutas, vehículos, operadores, costos, mantenimiento, riesgos y patrones, facilitando decisiones basadas en datos.

¿Por qué utilizar ETL?

Porque los datos originales deben prepararse antes de ser utilizados para análisis.

Extraer

↓

Limpiar

↓

Transformar

↓

Validar

↓

Cargar

¿Por qué utilizar un Data Warehouse?

Porque permite centralizar información preparada para análisis y consultas.

¿Por qué SQLite?

Porque proporciona almacenamiento relacional local sin requerir un servidor independiente para este proyecto.

Además, el dashboard trabaja directamente con el archivo:

data_warehouse\siloglog_dw.db

¿Por qué Streamlit?

Porque permite integrar Python, consultas, modelos y visualizaciones en una aplicación interactiva.

La implementación actual utiliza Streamlit como capa de presentación.

¿Qué diferencia hay entre clasificación y regresión?

Clasificación:

Predice una categoría.

Regresión:

Predice un valor numérico.

¿Qué es K-Means?

Es un algoritmo de agrupación que permite identificar registros con características similares.

¿Qué es PCA?

Es una técnica de reducción de dimensionalidad que permite estudiar las principales dimensiones de variación presentes en los datos.

¿Por qué guardar modelos .pkl?

Para reutilizar modelos ya entrenados sin ejecutar nuevamente todo el proceso de entrenamiento.

¿Por qué guardar el preprocesador?

Para garantizar que los datos de predicción reciban las mismas transformaciones aplicadas durante el entrenamiento.

¿Qué pasa si falta el Data Warehouse?

El dashboard detecta que no existe y detiene la ejecución mostrando el error correspondiente.

¿Qué pasa si no existen entregas?

El dashboard muestra un error y detiene el procesamiento para evitar continuar con información incompleta.

¿Qué pasa si una gráfica está vacía?

Primero deben revisarse:

1. Filtros.
2. Periodo.
3. Datos.
4. Columnas.
5. Consulta.

No deben inventarse valores.

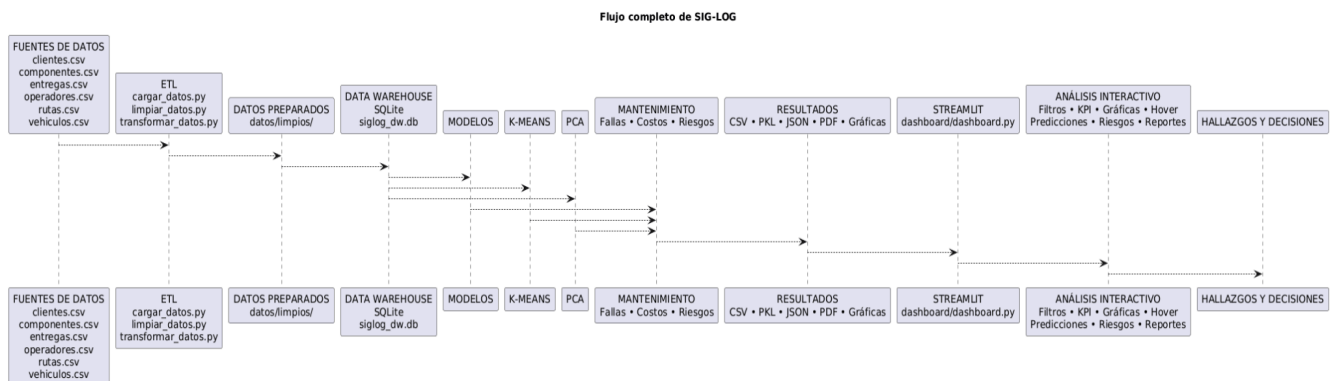
91. GLOSARIO

Término	Definición
CSV	Archivo tabular utilizado como fuente de datos
DataFrame	Estructura tabular de Pandas
ETL	Extracción, transformación y carga
Data Warehouse	Almacenamiento preparado para análisis
SQLite	Motor de base de datos relacional local
Dashboard	Interfaz interactiva de visualización
KPI	Indicador clave de desempeño
Feature	Variable de entrada de un modelo
Target	Variable objetivo
Clasificación	Predicción de categorías
Regresión	Predicción de valores numéricos

K-Means	Algoritmo de agrupación
Cluster	Grupo de registros similares
PCA	Análisis de componentes principales
Preprocesador	Transformaciones aplicadas antes del modelo
Modelo serializado	Modelo guardado para reutilización
Tooltip	Información mostrada al pasar el cursor
Traceback	Detalle técnico de una excepción
Insight	Hallazgo derivado de datos
Repositorio	Proyecto administrado mediante Git

92. ARQUITECTURA FINAL DEL SISTEMA

La arquitectura completa de SIG-LOG queda representada de la siguiente manera:



93. CICLO COMPLETO DE SIG-LOG

La filosofía técnica del proyecto puede resumirse en:

DATOS



INFORMACIÓN



ANÁLISIS



PATRONES



HALLAZGOS



DECISIONES



ACCIONES



VERIFICACIÓN

El sistema conecta información operativa con preguntas de negocio y permite pasar de registros a resultados interpretables.

94. CONCLUSIÓN TÉCNICA

SIG-LOG constituye una solución integral de análisis logístico que conecta diferentes etapas del procesamiento de datos en una arquitectura organizada.

El sistema parte de archivos fuente y aplica procesos de extracción, limpieza y transformación. Los datos preparados son posteriormente utilizados para alimentar un Data Warehouse basado en SQLite, desde donde se realizan consultas analíticas.

Sobre esta información se ejecutan diferentes técnicas de análisis, incluyendo:

- Clasificación.
- Regresión.
- K-Means.
- PCA.
- Análisis de mantenimiento.
- Análisis de riesgos.

Los modelos entrenados se conservan mediante archivos serializados y cuentan con un preprocesador específico para mantener consistencia entre entrenamiento y predicción.

Los resultados se integran posteriormente en un dashboard desarrollado con Streamlit, que actualmente contempla módulos de resumen, entregas, clientes, vehículos, operadores, rutas, combustible, asignaciones, mantenimiento, riesgos, preguntas de negocio, patrones, análisis avanzado, mapa operativo, reportes y datos.

La implementación actual también incorpora mecanismos de validación para evitar que el dashboard continúe cuando el Data Warehouse no está disponible o cuando no se pueden cargar las entregas.

Por lo tanto, SIG-LOG debe entenderse no como una colección aislada de gráficas, sino como un **pipeline completo de conocimiento**:

DATOS



ETL



DATOS LIMPIOS



DATA WAREHOUSE



ANALÍTICA



MODELOS



RESULTADOS



DASHBOARD



HALLAZGOS



DECISIONES



VERIFICACIÓN

El objetivo técnico final es conservar la **trazabilidad, reproducibilidad, mantenibilidad y consistencia** de todo el proceso.

De esta forma, otra persona puede recibir el proyecto, instalar el entorno, localizar los datos, comprender la arquitectura, ejecutar el dashboard, revisar los modelos, interpretar los resultados, diagnosticar problemas y continuar con el mantenimiento sin depender exclusivamente del desarrollador original.

ANEXO A. COMANDOS ESENCIALES

Crear entorno

conda create -n siglog python=3.13 -y

Activar entorno

conda activate siglog

Instalar dependencias

pip install streamlit pandas numpy plotly scikit-learn joblib matplotlib

Verificar Python

python --version

Ejecutar dashboard

streamlit run dashboard\dashboard.py

Ejecutar en otro puerto

streamlit run dashboard\dashboard.py --server.port 8502

Detener

Ctrl + C

Git

git status

git diff

git add .

git commit -m "Actualización SIG-LOG"

git push

ANEXO B. ESTRUCTURA DE ENTREGA

La entrega final debe conservar:

SIG-LOG/

|

|— README.md

|— requirements.txt

|

- |— clientes.csv
- |— componentes.csv
- |— entregas.csv
- |— operadores.csv
- |— rutas.csv
- |— vehiculos.csv
- |
- |— dashboard/
- |— datos/
- |— etl/
- |— data_warehouse/
- |— modelos/
- |— modelos_entrenados/
- |— mantenimiento/
- |— no_supervisado/
- |— pca/
- |— graficas/
- |— reportes/
- |— reportes_modelos/